

Phát triển ứng dụng web

JavaScript
(nâng cao)

Nội dung

- ☐ Regular Expression
- ☐ JQuery
- ☐ ES6
- ☐ Modules
- ☐ JS Debugging
- ☐ Asynchronous
- ☐ AJAX

Nội dung

- ☐ **Regular Expression**
- ☐ JQuery
- ☐ ES6
- ☐ Modules
- ☐ JS Debugging
- ☐ Asynchronous
- ☐ AJAX

Regular Expression

- ☐ **Regular Expression** là một ngôn ngữ cực mạnh dùng mô tả văn bản cũng như thao tác trên văn bản.
- ☐ **Regular Expression** là kỹ thuật xác định một chuỗi các ký tự sẽ được sử dụng trong mẫu tìm kiếm. **Regular Expression** được viết theo một cú pháp cụ thể và sau đó thường được áp dụng trên một chuỗi văn bản lớn hơn để xem chuỗi có đáp ứng các điều kiện được xác định trong **Regular Expression** hay không.

Syntax

- ❑ Regular expression có cú pháp chung với 1 **pattern** và **modifier** như sau:

`/pattern/modifier(s)`

- ❑ **Pattern** là chuỗi ký tự liên tiếp.
- ❑ **Modifier** là 1 ký tự xác định cách xử lý của **regular expression**

Cấu trúc Regular Expression

- ❑ Cơ bản bao gồm 2 thành phần là:
 - ❑ **Literal** (trực tiếp): đại diện cho ký tự cần so khớp (a, b, ..., Z, 0, 1, ..., 9, ...).
 - ❑ **Meta characters** (siêu ký tự): là ký tự đặc biệt (\, -, [,], (,), ^, \$, ...) hoạt động như chỉ thị lệnh trong regular expression.
- ❑ Ví dụ kiểm tra toàn số: `/^\d*$/`

Ví dụ

```
let str = "Mon hoc phat trien ung dung web";  
let regex = /o/;  
console.log(str.match(regex));
```



```
[  
  "o",  
  index: 1,  
  input: "Mon hoc phat trien ung dung web",  
  groups: undefined  
]
```

Sử dụng Meta Characters

```
let str = "Mon hoc phat trien ung dung web";  
let regex = /.e./;  
console.log(str.match(regex));
```



```
[  
  "ien",  
  index: 15,  
  input: "Mon hoc phat trien ung dung web",  
  groups: undefined  
]
```

Bộ Metacharacters cơ bản

Ký tự	Ý nghĩa
.	đại diện cho 1 ký tự bất kỳ trừ ký tự xuống dòng \n
\d	ký tự chữ số tương đương [0-9]
\D	ký tự ko phải chữ số
\s	ký tự khoảng trắng tương đương [\f\n\r\t\v]
\S	ký tự không phải khoảng trắng tương đương [^\f\n\r\t\v]
\w	ký tự word (gồm chữ cái và chữ số, dấu gạch dưới _) tương đương [a-zA-Z_0-9]
\W	ký tự không phải ký tự word tương đương [^a-zA-Z_0-9]
^	bắt đầu 1 chuỗi hay 1 dòng
\$	kết thúc 1 chuỗi hay 1 dòng
\A	bắt đầu 1 chuỗi
\Z	Kết thúc 1 chuỗi

Ký tự	Ý nghĩa
	ký tự ngăn cách so trùng tương đương với phép or (lưu ý cái này nếu muốn kết hợp nhiều điều kiện)
[abc]	khớp với 1 ký tự nằm trong nhóm là a hay b hay c
[a-z]	so trùng với 1 ký tự nằm trong phạm vi a-z, dùng dấu – làm dấu ngăn cách
[^abc]	sẽ không so trùng với 1 ký tự nằm trong nhóm, ví dụ không so trùng với a hay b hay c
()	Xác định 1 group (biểu thức con) xem như nó là một yếu tố đơn lẻ trong pattern, ví dụ ((a(b)c) sẽ khớp với b, ab, abc.
?	khớp với đứng trước từ 0 hay 1 lần. Ví dụ A?B sẽ khớp với B hay AB.
*	khớp với đứng trước từ 0 lần trở lên. A*B khớp với B, AB, AAB,...
+	khớp với đứng trước từ 1 lần trở lên. A+B khớp với AB, AAB,..
{n}	n là số, khớp đúng với n ký tự đứng trước nó. Ví dụ A{2} khớp đúng với AA.
{n,}	khớp đúng với n ký tự trở lên đứng trước nó, A{2,} khớp với AA, AAA,...
{m,n}	khớp đúng với từ m -> n ký tự đứng trước nó, A{2,4} khớp với AA,AAA,AAAA.

Sử dụng Pattern Modifiers

```
let str = "Mon hoc phat trien Ung dung web";  
let regex = /ung/gi;  
console.log(str.match(regex));
```



```
(2) ["Ung", "ung"]  
  1. 0: "Ung"  
  2. 1: "ung"
```

Các modifier

- ❑ **i**: không quan tâm hoa thường (case-insensitive)
- ❑ **g**: tìm kiếm toàn bộ, không dừng lại khi đã có giá trị đầu tiên so khớp thành công.
- ❑ **m**: tìm kiếm trên nhiều dòng.

Ví dụ so khớp email

```
let arr = ['matuan@fit.hcmus.edu.vn',  
          'matuan@gmail.com',  
          'matuan.yahoo.com',  
          'matuan@.com'];  
let regex = /^[a-z][a-z0-9_\.]{5,32}@[a-z0-9]{2,}(\.[a-z0-9]{2,}){1,3}$/;  
for (let email of arr) {  
  console.log(`Check: ${email} -> ` + regex.test(email));  
}
```



```
Check: matuan@fit.hcmus.edu.vn -> true  
Check: matuan@gmail.com -> true  
Check: matuan.yahoo.com -> false  
Check: matuan@.com -> false
```

Nội dung

- ☐ Regular Expression
- ☒ **jQuery**
- ☐ ES6
- ☐ Modules
- ☐ JS Debugging
- ☐ Asynchronous
- ☐ AJAX

JQuery

- ❑ **JQuery** là 1 thư viện JavaScript, mục tiêu giúp cho việc sử dụng javascript dễ dàng hơn với việc “write less, do more”.
- ❑ Các lợi ích JQuery đem lại là:
 - ❑ DOM Traversal and Manipulation
 - ❑ Event handling
 - ❑ AJAX

Ngắn gọn và dễ đọc

- ❑ JavaScript thuần:

```
// Tìm tất cả các thẻ p và thêm class 'highlight'
const paragraphs = document.querySelectorAll('p');
for (let i = 0; i < paragraphs.length; i++) {
  paragraphs[i].classList.add('highlight');
}
```

- ❑ JQuery:

```
// Sử dụng jquery để thực hiện tương tự
$('p').addClass('highlight');
```


Sử dụng

☐ Sử dụng CDN (Content Delivery Network) - Khuyến dùng

```
<script  
src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js">  
</script>
```

☐ Tải về local

- ☐ Truy cập link CDN kể trên hoặc <https://releases.jquery.com>
- ☐ Cung cấp qua server hoặc local

```
<script src="/js/jquery-3.7.1.min.js"></script>
```

Syntax và Document Ready

☐ Syntax:

```
$(selector).action()
```

☐ Sự kiện Document Ready

```
$(document).ready(function () {  
    // code xử lý khi DOM đã sẵn sàng  
});  
// Hoặc cách viết ngắn gọn hơn  
$(function () {  
    // code xử lý khi DOM đã sẵn sàng  
});
```



```
document.addEventListener("DOMContentLoaded", function () {  
    // code xử lý khi DOM đã sẵn sàng  
});
```

Selectors

☐ Sử dụng CSS selector

Loại selector	Ví dụ
Cơ bản	<code>\$("*")</code> , <code>\$("#myId")</code> , <code>\$(".myClass")</code> , <code>\$("p")</code> , <code>\$("p, div, .myClass")</code>
Theo thuộc tính	<code>\$("[href]")</code> , <code>\$("a[target='_blank']")</code>
Pseudo-selectors	<code>\$("tr:odd")</code> , <code>\$("p:first")</code> , <code>\$(":button")</code>

Di chuyển trong DOM (DOM Traversal)

☐ Hỗ trợ đầy đủ các phương thức hỗ trợ đến các phần tử liên quan.

☐ Đi lên (Ancestors):

- ☐ `.parent()`: Lấy cha trực tiếp.
- ☐ `.parents()`: Lấy tất cả các cha.

☐ Đi xuống (Descendants):

- ☐ `.children()`: Lấy tất cả con trực tiếp.
- ☐ `.find(selector)`: Tìm các phần tử con cháu khớp với selector.

☐ Ngang hàng (Siblings):

- ☐ `.siblings()`: Lấy tất cả anh em.
- ☐ `.next()`, `.prev()`.

Lấy và thay đổi nội dung

- ❑ Cung cấp các phương thức (get, set) đơn giản và dễ sử dụng.
- ❑ `.text( [value] )`: Lấy hoặc thiết lập nội dung dạng văn bản.
- ❑ `.html( [value] )`: Lấy hoặc thiết lập nội dung dạng HTML.
- ❑ `.val( [value] )`: Lấy hoặc thiết lập giá trị (value) của các phần tử trong form (<input>, <textarea>).

```
$("#myDiv").text("Hello!"); // Set text
let content = $("#myDiv").html(); // Get HTML content
$("#nameInput").val("John Doe"); // Set input value
```

Thao tác với Thuộc tính & CSS

- ❑ Thuộc tính (Attributes):
 - ❑ `.attr('attributeName')`: Lấy giá trị thuộc tính.
 - ❑ `.attr('attributeName', 'value')`: Thiết lập giá trị thuộc tính.
 - ❑ `.removeAttr('attributeName')`: Xóa thuộc tính.
- ❑ CSS:
 - ❑ `.css('propertyName')`: Lấy giá trị thuộc tính CSS.
 - ❑ `.css('propertyName', 'value')`: Thiết lập thuộc tính CSS.
 - ❑ `.css({ "color": "red", "font-size": "20px" })`: Thiết lập nhiều thuộc tính cùng lúc.
- ❑ Classes:
 - ❑ `.addClass('className')`: Thêm một hoặc nhiều lớp.
 - ❑ `.removeClass('className')`: Xóa một hoặc nhiều lớp.
 - ❑ `.toggleClass('className')`: Bật/tắt một lớp. Nếu có thì xóa, nếu chưa có thì thêm.
 - ❑ `.hasClass('className')`: Kiểm tra xem phần tử có lớp đó không (trả về true/false).

Xử lý phần tử DOM

- ❑ `.append()`: Chèn nội dung vào cuối của phần tử được chọn.
- ❑ `.prepend()`: Chèn nội dung vào đầu của phần tử được chọn.
- ❑ `.before()`: Chèn nội dung trước phần tử được chọn.
- ❑ `.after()`: Chèn nội dung sau phần tử được chọn.
- ❑ `.remove()`: Xóa phần tử được chọn (và cả các con của nó).
- ❑ `.empty()`: Xóa tất cả các phần tử con của phần tử được chọn.

Event handling

- ❑ Syntax: `$(selector).on('eventName', function() { ... });`
- ❑ Cung cấp các phương thức ngắn

- ❑ `.click(function)`
- ❑ `.hover(functionIn, functionOut)`
- ❑ `.submit(function)`
- ❑ `.keydown(function)`

```
$("#myButton").click(function () { /*code*/ });
```



```
$("#myButton").on('click', function () { /*code*/ });
```



```
document.querySelector('#myButton').addEventListener('click', function () { /*code*/ });
```

this trong JQuery

- ❑ Bên trong một hàm xử lý sự kiện, từ khóa **this** tham chiếu đến phần tử DOM gốc đã kích hoạt sự kiện đó.
- ❑ Để sử dụng các phương thức JQuery trên nó thì cần bọc nó lại bằng **\$()** -> **\$(this)**.

```
$("#p").on('click', function () {  
    // this là thẻ p được click (DOM element)  
    // $(this) là đối tượng JQuery của thẻ p đó  
    $(this).css("color", "blue");  
});
```

Hiệu ứng (effect)

- ❑ Dễ dàng thêm các hiệu ứng động vào trang web.
- ❑ **.hide(speed, callback)**: Ẩn phần tử.
- ❑ **.show(speed, callback)**: Hiện thị phần tử.
- ❑ **.toggle(speed, callback)**: Bật/tắt hiển thị.
 - ❑ **speed** có thể là "slow", "fast", hoặc số mili giây (ví dụ: 1000).
 - ❑ **callback** là một hàm sẽ chạy sau khi hiệu ứng hoàn thành.

Nội dung

- ☐ *Regular Expression*
- ☐ *JQuery*
- ☐ **ES6**
- ☐ Modules
- ☐ JS Debugging
- ☐ Asynchronous
- ☐ AJAX

JavaScript ES6 (ECMAScript 2015)

- ☐ Template strings
- ☐ Default arguments
- ☐ Arrow Functions
- ☐ Const
- ☐ Object
- ☐ Classes

Template strings

```
<script type="module">
  import { arrInt, strName } from
    './js/moduleExport1.js';
  import { productName, productPrice } from
    './js/moduleExport1.js';

  let str = `<h1>${arrInt}</h1>
    <h2>${strName}</h2>
    <p>
      ${productName} -
      ${productPrice}
    </p>`;
  console.log(str);
</script>
```

Default Arguments (Parameters)

```
<script>
  function testDefault(x, y = 10) {
    return x + y;
  }

  let x = 18;
  console.log(testDefault(x));
</script>
```

Arrow Function

```
arrowFunc1 = (x, y) => { return x + y };  
console.log(arrowFunc1(2, 3));
```

```
arrowFunc2 = (x, y) => x + y;
```

```
arrowFunc3 = x => x * x;
```

```
arrowFunc4 = () => "0 args";
```

JavaScript Const

- ☐ Biến được khai báo với **const** thì không được phép khai báo lại (trong cùng **scope**).
- ☐ Biến được khai báo với **const** thì không được đặt lại giá trị. Do đó khi khai báo phải gán luôn giá trị.
- ☐ Biến được khai báo với **const** thì có **Block Scope** (giống **let**).
- ☐ Sử dụng **const** khi kiểu biến là:
 - ☐ Array
 - ☐ Object
 - ☐ Function
 - ☐ RegExp

JavaScript Objects

```
const product = {  
  Name: 'Product1',  
  Price: 100.0,  
  Quantity: 10,  
  toString: function() {  
    return `${this.Name} (${this.Price})`;  
  }  
};  
console.log(`Product: ${product.Name}  
(${product['Price']})`);  
console.log(product.toString());
```

Classes

- ❑ **JavaScript Classes** are *templates* for JavaScript Objects.
- ❑ **Syntax**
 - ❑ Từ khóa để khai báo: **class**
 - ❑ Luôn tạo method: **constructor()**

Ví dụ

```
class Fraction {  
  constructor(numerator, denominator) {  
    this.numerator = numerator;  
    this.denominator = denominator;  
  }  
  Plus(fOther) {  
    let rD = this.denominator * fOther.denominator;  
    let rN = this.numerator * fOther.denominator +  
              this.denominator * fOther.numerator;  
    return new Fraction(rN, rD);  
  }  
  Minus(fOther) {  
    // implement code  
    return new Fraction(rN, rD);  
  }  
}
```

```
let f1 = new Fraction(3, 5);  
let f2 = new Fraction(7, 11);  
console.log(f1.Plus(f2));
```

Nội dung

- ☐ Regular Expression
- ☐ JQuery
- ☐ ES6
- ☐ **Modules**
- ☐ JS Debugging
- ☐ Asynchronous
- ☐ AJAX

JavaScript Modules

- ❑ **Modules** giúp tách **code** ra thành nhiều **file**.
- ❑ Sử dụng **Export** và **Import** để tách các **hàm (function)** hoặc **biến (variable)** giữa các **file code**.

Named Exports

❑ Export

```
export const arrInt = [ 1, 2, 3 ];  
export const strName = 'Name';  
  
const productName = 'Product01';  
const productPrice = 120.5;  
  
export {productName, productPrice};
```

❑ Import

```
<script type="module">  
  import { arrInt, strName } from './js/moduleExport1.js';  
  import { productName, productPrice } from  
'./js/moduleExport1.js';  
  console.log(arrInt);  
  console.log(strName);  
  console.log(productName);  
  console.log(productPrice);  
</script>
```

Default Exports

❑ Export

```
const funcMsg = fullName => {  
  return `Hello ${fullName}!`;  
};  
  
export default funcMsg;
```

❑ Import

```
<script type="module">  
  import func from './js/moduleExport2.js';  
  console.log(func('Test'));  
</script>
```

Nội dung

- ❑ *Regular Expression*
- ❑ *JQuery*
- ❑ *ES6*
- ❑ *Modules*
- ❑ **JS Debugging**
- ❑ Asynchronous
- ❑ AJAX

JavaScript Debuggers

- ❑ **Code Debugging** là công việc tìm các lỗi để sửa trong chương trình.
- ❑ Tất cả các trình duyệt hiện đại đều được tích hợp sẵn **JavaScript debugger** giúp thực hiện debug với code JS.
- ❑ Sử dụng **debugger keyword** để tạo **breakpoint** và kích hoạt chức năng **debug**, nếu môi trường không hỗ trợ debug thì **debugger** sẽ được bỏ qua.

```
<script>
  let v = 3 * 10;
  debugger;
  console.log(v);
</script>
```

JavaScript Errors

```
try {
  // các dòng lệnh có khả năng phát sinh lỗi runtime
  throw 'msg'; // tự quăng lỗi dạng string
  throw 501;   // tự quăng lỗi dạng number
}
catch (err) {
  // các xử lý dựa vào thông tin lỗi trong error object err
  throw 'error'; // ?
}
finally {
  // khối lệnh này luôn thực thi bất chấp kết quả của try / catch
}
```

Nội dung

- ☐ *Regular Expression*
- ☐ *JQuery*
- ☐ *ES6*
- ☐ *Modules*
- ☐ *JS Debugging*
- ☐ **Asynchronous**
- ☐ AJAX

JavaScript Async

- ☐ Callbacks
- ☐ Timer
- ☐ Promise
- ☐ Async / Await

JavaScript Callbacks

- ❑ Một **callback** là 1 **function** truyền như là 1 **tham số** của 1 **function** khác.
- ❑ Sử dụng **callback** trong xử lý **bất đồng bộ** (**asynchronous**) khi mà 1 **function** phải chờ kết quả thực hiện của **function** khác.

Timer - setTimeout

```
function doIt() {  
    console.log('Time now:', (new Date()).toLocaleTimeString());  
}  
  
doIt();  
let t = setTimeout(doIt, 5000);  
setTimeout(doIt, 2000);  
clearTimeout(t);
```



```
Time now: 9:19:28 PM  
Time now: 9:19:30 PM
```

Timer - setInterval

```
let s = 0;
let t = setInterval(() => {
  console.log('Time now:', (new Date()).toLocaleTimeString());
  s++;
  if(s > 5) {
    clearInterval(t);
  }
}, 1000);
```



```
Time now: 9:43:30 PM
Time now: 9:43:31 PM
Time now: 9:43:32 PM
Time now: 9:43:33 PM
Time now: 9:43:34 PM
Time now: 9:43:35 PM
Time now: 9:43:36 PM
```

Asynchronous - Callback

```
function sleep(t) {
  const timeUp = (new Date()).getTime() + t;
  while((new Date()).getTime() < timeUp);
}
function doChore(chore, callback) {
  console.log(`Started ${chore}...`);
  sleep(2000);
  callback();
}
function finish() {
  console.log("Finished my chore!");
}
function run() {
  doChore('task-01', finish);
  console.log('task-02');
}
```



```
Started task-01...
Finished my chore!
task-02
```


Asynchronous - Promise

- ❑ **JavaScript Promise** là **object** chứa cả **producing code** và lời gọi đến **consuming code**.
- ❑ **JavaScript Promise object** bao gồm 2 **property**: **state** và **result**.

state	result
"pending"	undefined
"fulfilled"	a result value
"rejected"	an error object

Ví dụ

```
function doChorePromise(chore){
  return new Promise((resolve, reject) => {
    console.log(`Started ${chore}...`);
    sleep(2000);
    if(chore.length > 0){
      resolve(`Finished my ${chore}!`);
    } else{
      reject('no task');
    }
  });
}

function run(){
  doChoreAsync('task-01').then(resolve => {
    console.log(resolve.message);
  }).catch(error => {
    console.log(reject.message);
  });
  console.log('task-02');
}
```

Asynchronous - Async

- ❑ **async** dùng phía trước **function** để khai báo **function** trả về 1 **promise**

```
async function doChoreAsync(chore){
  console.log(`Started ${chore}...`);
  sleep(2000);
  if (chore.length > 0){
    return `Finished my ${chore}!`;
  } else{
    throw new Error('no task');
  }
}

function run(){
  doChoreAsync('task-01').then(resolve => {
    console.log(resolve.message);
  }).catch(error => {
    console.log(reject.message);
  });
  console.log('task-02');
}
```

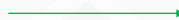
Asynchronous - Await

- ❑ **await** dùng phía trước lời gọi **promise** để khai báo **function** chờ kết quả của 1 **promise**
- ❑ **await** chỉ được phép sử dụng trong 1 **async function**

```
async function run(){
  try{
    const result = await doChoreAsync('task-01');
    console.log(result);
  } catch(error){
    console.log(error);
  }
  console.log('task-02');
}
```



```
Started task-01...
task-02
Finished my task-01!
```



```
Started task-01...
Finished my task-01!
task-02
```

Nội dung

- ☐ Regular Expression
- ☐ JQuery
- ☐ ES6
- ☐ Modules
- ☐ JS Debugging
- ☐ Asynchronous
- ☐ **AJAX**

AJAX – JSON (JavaScript Object Notation)

```
const jsData = `[{ "name": "ABC", "age": 30, "city": "HCM"},  
  { "name": "MNH", "age": 23, "city": "HN"},  
  { "name": "WER", "age": 36, "city": "DN"}  
];  
  
const ds = JSON.parse(jsData);  
console.log('object: ', ds);  
const str = JSON.stringify(ds);  
console.log('string: ', str);
```



```
object: - [{name: "ABC", age: 30, city: "HCM"}, {name: "MNH", age: 23, city:  
  "HN"}, {name: "WER", age: 36, city: "DN"}] (3)  
string: - "[{"name":"ABC","age":30,"city":"HCM"},{"name":"MNH","age":23,"city":  
  "HN"},{"name":"WER","age":36,"city":"DN"}]"
```

AJAX - XMLHttpRequest

```
const xhr = new XMLHttpRequest();

xhr.onload = function() {
  if (this.status >= 200 && this.status < 300) {
    $('#content').html(this.responseText);
    let obj = JSON.parse(this.responseText);
    console.log(obj);
  }
};

xhr.onerror = function () {
  console.log(new Error({
    status: this.status,
    statusText: this.statusText
  }));
};

xhr.open("GET", "http://localhost:3000", true);
xhr.send();
```

AJAX - fetch

```
fetch('http://localhost:3000').then(response => {
  response.text().then(str => {
    $('#content').html(str);
    const obj = JSON.parse(str);
    console.log(obj);
  });
});
```



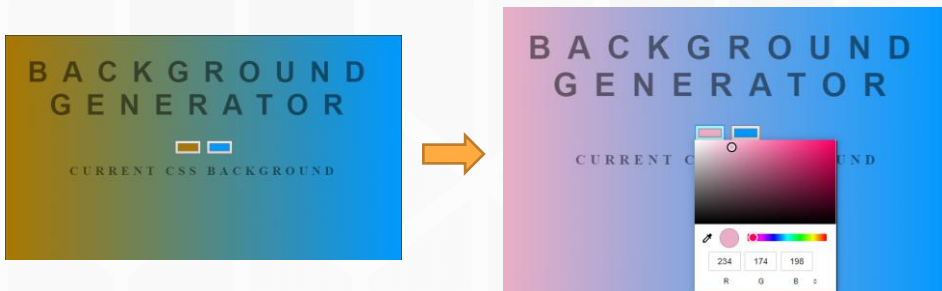
```
const response = await fetch('http://localhost:3000');
const str = await response.text();
$('#content').html(str);
const obj = JSON.parse(str);
console.log(obj);
```

Nội dung

- ☐ *Regular Expression*
- ☐ *JQuery*
- ☐ *ES6*
- ☐ *Modules*
- ☐ *JS Debugging*
- ☐ *Asynchronous*
- ☐ *AJAX*
- ☐ **Bài tập**

Bài tập

- ☐ Xây dựng trang web tạo màu nền đơn giản (cho phép lựa chọn màu realtime) như sau:



Bài tập

- ❑ Với dữ liệu lấy được từ address: '<https://reqres.in>' xây dựng trang HTML sử dụng ajax trình bày dữ liệu như hình dưới

